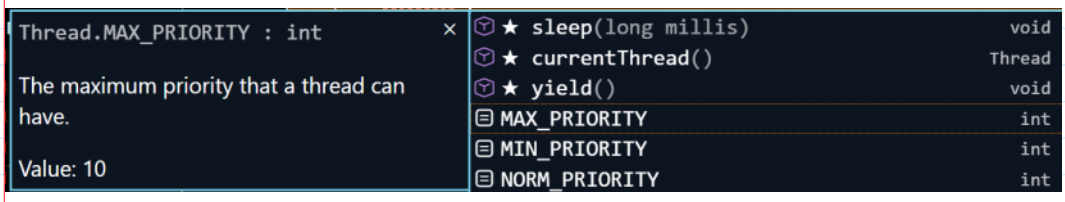
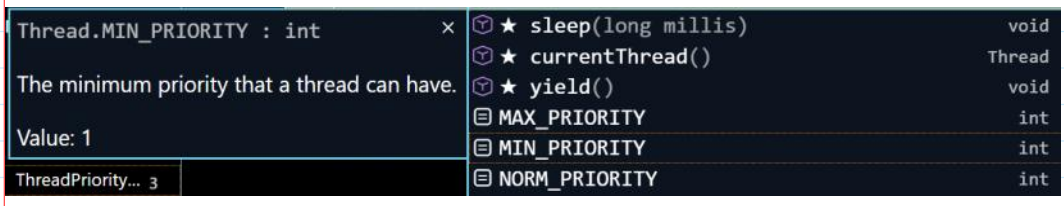
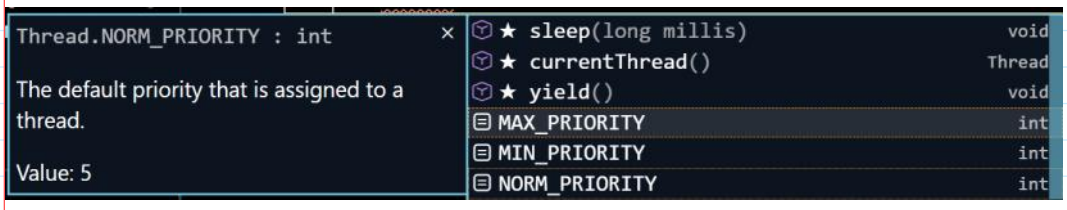
Thread Priorities

30 November 2025 10:20

Every Thread in Java has some priority it may be default priority generated by JVM or customized priority provided by the programmer. The valid range of priority is 1 to 10, where 1 is the minimum priority and 10 is the maximum priority. Thread class defines the following constants to represent some standard priority:





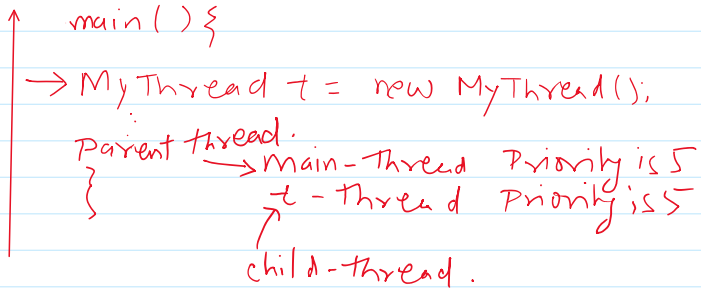


Thread schedular will use priorities while allocating processor time. The thread which is having highest priority will get chance first. If two threads having same priority then we can't expect the exact execution of threads. It depends upon Thread Schedular.

```
public final int getPriority()
public final void setPriority(int p)
```

```
t.setPriority(7);
t.setpriority(17); //Error: RE: IllegalArgumentException
```

NORM_PRIORITY(default priority): The NORM_PRIORITY is 5 or the priority of main-thread is 5. But for all remaining threads NORM_PRIORITY is inherited from parent to child, i.e., whatever priority parent thread has, the priority will be there for child thread.




```
package MyThreads;
public class FirstThread implements Runnable {
    private String threadName;
```

```java
    public FirstThread(String threadName) {
        this.threadName = threadName;
    }

    @Override
    public void run(){
        final int max = 10;
        for (int i = 0; i < max; i++) {
            System.out.println(i+": Child thread = "+ threadName);
            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}


package MyThreads;
public class ThreadPriority {
    public static void main(String[] args) throws InterruptedException {
        FirstThread ft1 = new FirstThread("ft1");
        FirstThread ft2 = new FirstThread("ft2");
        Thread t2 = new Thread(ft2);
        t2.setPriority(Thread.MAX_PRIORITY);
        Thread t1 = new Thread(ft1);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        t2.start();
        Thread.sleep(2000);
        for(int i = 0; i < 10; i++ ){
            System.out.println("\nMain thread");
        }
    }
}
```

Output:

```
  0: Child thread = ft2
  1: Child thread = ft2
  2: Child thread = ft2
  3: Child thread = ft2
  4: Child thread = ft2
  5: Child thread = ft2
  6: Child thread = ft2
  7: Child thread = ft2
  8: Child thread = ft2
  9: Child thread = ft2
  0: Child thread = ft1
  1: Child thread = ft1
  2: Child thread = ft1
  3: Child thread = ft1
  4: Child thread = ft1
  5: Child thread = ft1
  6: Child thread = ft1
  7: Child thread = ft1
  8: Child thread = ft1
  9: Child thread = ft1
```

```
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
```